

Reviewer Pack — Block Design Discrepancy and Communication Complexity Lower ...

Entry 8a6982e09855 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 08:32:43 UTC

Block Design Discrepancy and Communication Complexity Lower Bound

Entry ID: 8a6982e09855 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-12 08:32:43 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Design Theory **Field B** (complexity object): Communication Complexity (Discrepancy of Set Disjointness)

Statement:

For a symmetric (v, k, λ) block design, the discrepancy of the communication matrix for the set disjointness function is $\Omega(\sqrt{\lambda v / k})$

Rationale (proposer's reasoning):

Block designs impose regular structure on incidence relations, which may constrain the discrepancy of associated communication matrices, offering a combinatorial lens to bound communication complexity

Taxonomy category: COMM_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 148d9e6a18826b36

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def generate_symmetric_block_design(v, k, lambda_):
    if v * (v - 1) % (2 * k) != 0:
        return None
    blocks = []
    points = list(range(v))
    for _ in range(k):
        block = random.sample(points, k)
        if all(len(set(block).intersection(other)) == lambda_ for other in blocks):
            blocks.append(block)
        else:
            return None
    return blocks

def construct_communication_matrix(blocks):
    n = len(blocks)
    matrix = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i + 1, n):
            common_points = set(blocks[i]).intersection(blocks[j])
            matrix[i][j] = matrix[j][i] = len(common_points)
    return matrix

def discrepancy(matrix):
    n = len(matrix)
    max_discrepancy = 0
    for i in range(n):
        row_sum = sum(matrix[i])
        col_sum = sum(matrix[j][i] for j in range(n))
        max_discrepancy = max(max_discrepancy, abs(row_sum - col_sum))
    return max_discrepancy

def run_trial(seed: int) -> dict:
    random.seed(seed)
    v = random.choice([10, 15, 20, 30, 40])
    k = random.randint(2, v // 2)
    lambda_ = random.randint(1, min(k - 1, v - k))
    design = generate_symmetric_block_design(v, k, lambda_)
    if not design:
        return {
            "metric_name": "discrepancy",
            "metric_value": None,
            "instances_tested": 0,
```


8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

All trials report instances_tested=0, indicating no empirical evaluation was performed. The 'design_not_possible' counterexample is a logical contradiction rather than an empirical failure. The conjecture's validity depends on existence of symmetric block designs, which the test neither verifies nor measures.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

No empirical tests were executed (instances_tested=0), and the counterexample is a logical contradiction rather than an empirical failure. | next: Implement tests with concrete symmetric block design parameters and measure discrepancy values directly

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	101607
2	preregistration	ollama_remote	qwen3:8b	0	0	24056
3	novelty	ollama_remote	qwen3:8b	0	0	20519
4	novelty	ollama_remote	qwen3:8b	0	0	15220
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17602
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9188
7	critic	ollama_remote	qwen3:8b	0	0	26801
8	judge	ollama_remote	qwen3:8b	0	0	12852

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 227845 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/8a6982e09855.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/8a6982e09855.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/8a6982e09855.tar.gz (if generated)